

Aula 24 - Consumo de APIs REST com Axios no Vue.js

Duração: 4 horas (1h teórica + 3h prática)

Parte Teórica (1 hora)

1. Revisão da Aula Anterior (15 min)

- Discussão sobre:
 - Composables e reutilização de lógica.
 - Dúvidas sobre `useFetch` personalizado.

2. Axios vs Fetch API (20 min)

Característica	Axios	Fetch API
Instalação	<code>npm install axios</code>	Nativo (não requer instalação)
Transformação de Dados	Automática (JSON)	Manual (<code>response.json()</code>)
Interceptores	Suporte nativo	Requer implementação manual
Cancelamento	<code>CancelToken</code>	<code>AbortController</code>

3. Padrões Recomendados (25 min)

Estrutura de Pastas

```
plaintext

src/
├─ api/
│   └─ http.js           # Config Axios
│   └─ userService.js    # Camada de serviço
└─ composables/          # (Opcional) useApi
```

Interceptores Úteis

```
javascript
```

```
// http.js
axios.interceptors.request.use(config => {
  const token = localStorage.getItem('token')
  if (token) config.headers.Authorization = `Bearer ${token}`
  return config
})
```

Parte Prática (3 horas)

Atividade: Sistema de Posts com API JSONPlaceholder

Passo 1: Configuração do Axios (45 min)

1. Instalação:

```
bash

npm install axios
```

2. Configuração Base (api/http.js):

```
javascript

import axios from 'axios'

const http = axios.create({
  baseURL: 'https://jsonplaceholder.typicode.com',
  timeout: 5000
})

export default http
```

3. Serviço de Posts (api/postService.js):

```
javascript

import http from './http'

export const PostService = {
  async getAll() {
    const response = await http.get('/posts')
    return response.data
  },
  async create(post) {
    const response = await http.post('/posts', post)
    return response.data
  }
}
```

```
}  
}
```

Passo 2: Integração com Componente (1h)

1. Listagem de Posts (`PostList.vue`):

html

```
<template>  
  <div>  
    <button @click="loadPosts">Carregar Posts</button>  
    <ul v-if="!loading">  
      <li v-for="post in posts" :key="post.id">{{ post.title }}</li>  
    </ul>  
    <p v-else>Carregando...</p>  
  </div>  
</template>  
  
<script setup>  
import { ref } from 'vue'  
import { PostService } from '@api/postService'  
  
const posts = ref([])  
const loading = ref(false)  
  
const loadPosts = async () => {  
  loading.value = true  
  posts.value = await PostService.getAll()  
  loading.value = false  
}  
</script>
```

2. Criação de Post (`PostForm.vue`):

html

```
<template>  
  <form @submit.prevent="handleSubmit">  
    <input v-model="post.title" placeholder="Título">  
    <textarea v-model="post.body"></textarea>  
    <button type="submit">Salvar</button>  
  </form>  
</template>  
  
<script setup>  
import { reactive } from 'vue'  
import { PostService } from '@api/postService'  
  
const post = reactive({ title: '', body: '' })
```

```
const handleSubmit = async () => {
  await PostService.create(post)
  alert('Post criado!')
  Object.assign(post, { title: '', body: '' })
}
</script>
```

Passo 3: Tratamento de Erros (1h15min)

1. Interceptor Global (api/http.js):

javascript

```
http.interceptors.response.use(
  response => response,
  error => {
    if (error.response.status === 401) {
      window.location.href = '/login'
    }
    return Promise.reject(error)
  }
)
```

2. Feedback no Componente:

javascript

```
const loadPosts = async () => {
  try {
    loading.value = true
    posts.value = await PostService.getAll()
  } catch (error) {
    console.error('Falha ao carregar posts:', error)
  } finally {
    loading.value = false
  }
}
```

3. Composable useApi (Opcional):

javascript

```
// composables/useApi.js
import { ref } from 'vue'
import { PostService } from '@api/postService'

export function useApi() {
  const error = ref(null)

  const fetchPosts = async () => {
```

```
    try {
      return await PostService.getAll()
    } catch (err) {
      error.value = err
      throw err
    }
  }

  return { error, fetchPosts }
}
```

Tarefa para Casa

1. Implementar Atualização/Exclusão:

- Adicionar métodos `update` e `delete` no `PostService`.

2. Cache Local:

- Usar Pinia para armazenar posts já carregados.

Próxima Aula

- **Tópico:** Autenticação JWT no Vue.js.
- **Atividade prática:** Integrar login com backend Spring Boot.

Resultado Esperado

- Aplicação Vue consumindo API REST com:
 - Configuração centralizada do Axios.
 - Tratamento de erros global.
 - Componentes reativos.

plaintext

- ✓ Chamadas HTTP organizadas em serviços
- ✓ Interceptores para headers/erros
- ✓ Pronto para autenticação JWT

https://miro.medium.com/v2/resize:fit:1400/1*JfJbA0kiLkD2w2Wm6u2VZQ.png (Fluxo de requisições com Axios) 🚀